CIS 3370 · INTRUSION DETECTION SYSTEMS · LAB C OF 3

Logstash structures a log with grok

Filebeat's modules know how to read logs that thousands of people share, like syslog and Zeek. Your own applications write logs nobody has a module for. Logstash is where you teach the stack to read them: a pipeline receives raw lines, a `grok` filter cuts each line into named fields, and the result goes to Elasticsearch with the right types. In this lab you build the pattern in Kibana's Grok Debugger, let Logstash take its pipelines from Kibana instead of from files, create the pipeline in **Stack Management → Logstash Pipelines**, and watch a mapping mistake happen and get fixed. Every number and screenshot here comes from running these steps on the class stack.

1. A log nobody has a module for
2. Build the pattern
3. Pipelines from Kibana
4. Create the pipeline
5. Ship and check
6. Fix the mapping

0 Before you start

Finish Lab A first. You need its header file and the two names `ES` and `KEYFILE` in your host terminal, and Filebeat installed on `client-01`. Lab B is not required.

This lab changes a configuration file the Logstash container may not change itself: `logstash/config/logstash.yml`, mounted read-only (Lab A, section 1). You edit it on the host. Make a copy first, so you can put it back at the end:

HOST

```
cp logstash/config/logstash.yml logstash/config/logstash.yml.original
```

1 A log nobody has a module for

CLIENT-01 Write thirty login records the way a small web application might. Each line is a time, a level, a user, an address, an action, a result and a duration. Every fourth one is `admin` failing.

```
cd /home/ubuntu/data/lab
users=(alice bob carol admin)
for i in $(seq 1 30); do
  u=${users[$((i % 4))]}
  lvl=INFO; res=success
  if [ "$u" = admin ]; then lvl=WARN; res=failure; fi
  ts=$(date -u +%Y-%m-%dT%H:%M:%SZ)
  echo "$ts $lvl $u 172.21.0.5 login $res $((20 + i))ms" >> app.log
done
wc -l app.log
head -3 app.log
grep -c failure app.log
```

```
30 app.log
...Z INFO bob 172.21.0.5 login success 21ms
...Z INFO carol 172.21.0.5 login success 22ms
...Z WARN admin 172.21.0.5 login failure 23ms
7
```

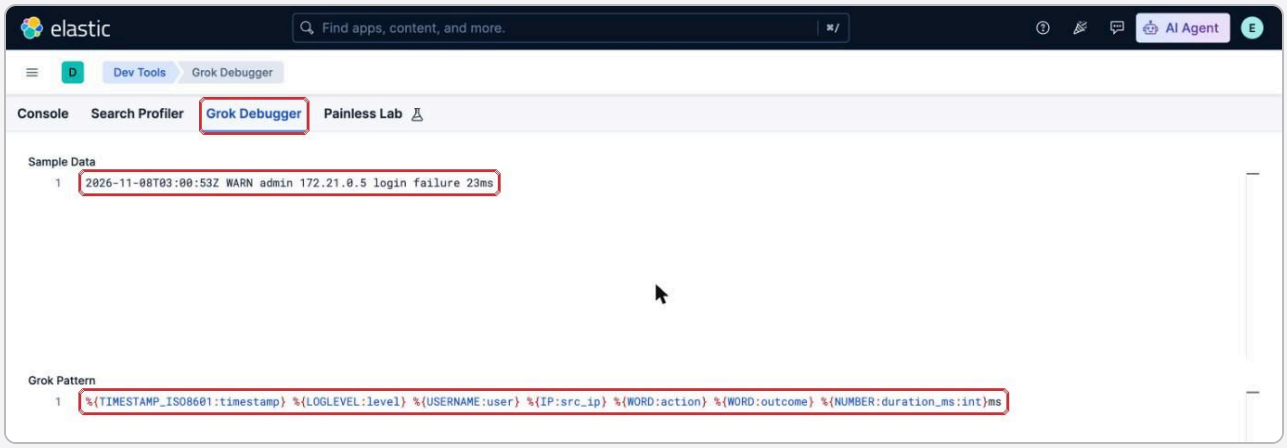
Thirty lines, seven failures. Those two numbers are what Elasticsearch must show at the end.

2 Build the pattern in the Grok Debugger

A grok pattern is the line, rewritten as named pieces. `%{IP:src_ip}` means "here is something shaped like an IP address; call it `src_ip`". `:int` on the end makes the value a number instead of text. Logstash ships with hundreds of named shapes: `TIMESTAMP_ISO8601`, `LOGLEVEL`, `USERNAME`, `WORD`, `NUMBER`.

KIBANA **Menu → Management → Dev Tools → Grok Debugger.** Paste one line from `app.log` into **Sample Data**, and this into **Grok Pattern**, all on one line:

```
%{TIMESTAMP_ISO8601:timestamp} %{LOGLEVEL:level} %{USERNAME:user} %
{IP:src_ip} %{WORD:action} %{WORD:outcome} %{NUMBER:duration_ms:int}ms
```



Grok Debugger: one sample line, and the pattern.

Click **Simulate**. Every piece comes back named, and `duration_ms` is the number `23`, not the text `"23"`.



Structured Data: seven fields from one line.

If a piece is wrong, the debugger returns nothing at all. Change one piece at a time until it matches; that is how every grok pattern is written.

3 Let Logstash take its pipelines from Kibana

docker-elk's Logstash reads its pipeline from `logstash/pipeline/logstash.conf` on the host. *Centralized pipeline management* makes it read pipelines from Elasticsearch instead, where Kibana's Logstash Pipelines screen stores them. Logstash needs an account to read them.

An account that can only read pipelines

> **KIBANA UI**
>
> As in Lab A, section 4: **Stack Management → Security → Roles → Create role** named `logstash_cpm` with cluster privileges `monitor` and `manage_logstash_pipelines`, no index privileges. Then **Users → Create user** `logstash_cpm` with that role and a password of your own.

CONSOLE

```
PUT _security/role/logstash_cpm
{
  "description": "Logstash may read the pipelines Kibana stores",
  "cluster": ["monitor", "manage_logstash_pipelines"]
}

PUT _security/user/logstash_cpm
{
  "password": "choose-your-own",
  "roles": ["logstash_cpm"],
  "full_name": "Logstash pipeline management"
}
```

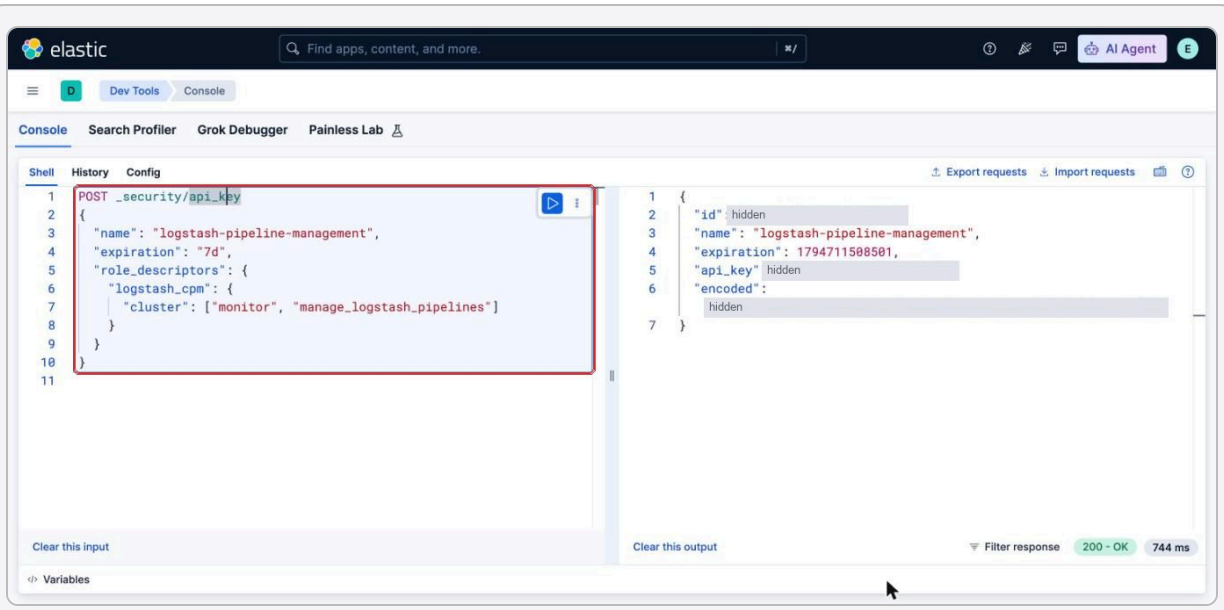
CURL

HOST

```
curl -s -H "$KEYFILE" -X PUT "$ES/_security/role/logstash_cpm" \
  -H 'Content-Type: application/json' \
  -d '{"cluster": ["monitor", "manage_logstash_pipelines"]}'
read -rsp "Password for logstash_cpm: " CPM_PW; echo
curl -s -H "$KEYFILE" -X PUT "$ES/_security/user/logstash_cpm" \
  -H 'Content-Type: application/json' \
  -d "{\"password\": \"$CPM_PW\", \"roles\": [\"logstash_cpm\"]}"
```

Keep the terminal open: the next step uses `CPM_PW` once more, then forgets it.

Why not an API key, as in Lab A? It was the first thing tried on the class stack. The key was created in the Console with a role descriptor limiting it to those two privileges:



An API key limited to two privileges. The secret values are hidden.

Logstash then refused to start:

```
ArgumentError: Using api_key authentication requires SSL/TLS secured
communication
```

Logstash will not send an API key over plain HTTP, and this stack has no TLS. A password from the Logstash keystore is the safe choice here. If you made such a key, revoke it as in Lab A.

The password goes in the Logstash keystore

The keystore lives inside the Logstash container, so Logstash must be running, still on its old settings, when you add to it. The keystore asks whether to continue without its own password; answer `y`.

HOST

```
printf 'y\n' | docker compose exec -T logstash bin/logstash-keystore create
printf '%s\n' "$CPM_PW" \
  | docker compose exec -T logstash bin/logstash-keystore add LS_CPM_PWD
unset CPM_PW
docker compose exec -T logstash bin/logstash-keystore list
```

```
Created Logstash keystore at /usr/share/logstash/config/logstash.keystore
Enter value for LS_CPM_PWD: Added 'ls_cpm_pwd' to the Logstash keystore.
ls_cpm_pwd
```

Logstash stores the name in lower case and finds it either way.

Edit logstash.yml on the host

`HOST` Open `logstash/config/logstash.yml` with nano or VS Code and add these lines at the end:

```
## Pipelines come from Kibana, not from logstash/pipeline/
xpack.management.enabled: true
xpack.management.elasticsearch.hosts: ["http://elasticsearch:9200"]
xpack.management.elasticsearch.username: "logstash_cpm"
xpack.management.elasticsearch.password: "${LS_CPM_PWD}"
xpack.management.pipeline.id: ["cis3370-applog"]
xpack.management.logstash.poll_interval: 5s
```

SETTING	WHAT IT DOES
`pipeline.id`	Which stored pipelines this Logstash runs. Only listed IDs run.
`poll_interval`	How often Logstash checks Kibana for changes
`${LS_CPM_PWD}`	Read from the Logstash keystore. The file holds no secret.

```
docker compose restart logstash
docker compose logs --since 1m logstash \
  | grep -E "configmanagement|configuration"
```

```
... Connected to ES instance {url:
"http://logstash_cpm:xxxxxx@elasticsearch:9200/"}
... No configuration found in the configured sources.
```

Connected, and nothing to run yet, because the pipeline does not exist. That message repeats every five seconds until you create it.

Two side effects to know about. With management on, Logstash ignores `logstash/pipeline/logstash.conf`, so docker-elk's default inputs on ports 5044 and 50000 are gone until you restore the file at the end. And the keystore lives in the container: if the Logstash container is recreated, add `LS_CPM_PWD` again.

4 Create the pipeline

The pipeline has three parts. **input**: listen for Filebeat on port 5044. **filter**: `grok` cuts the line, `date` makes the line's own time the event's time, and `mutate` renames the pieces to the standard ECS names. The pattern is the one from the debugger, split into two named parts with `pattern_definitions` so each line stays short. **output**: send to Elasticsearch as docker-elk's `logstash_internal` user, whose password Logstash already reads from its environment.

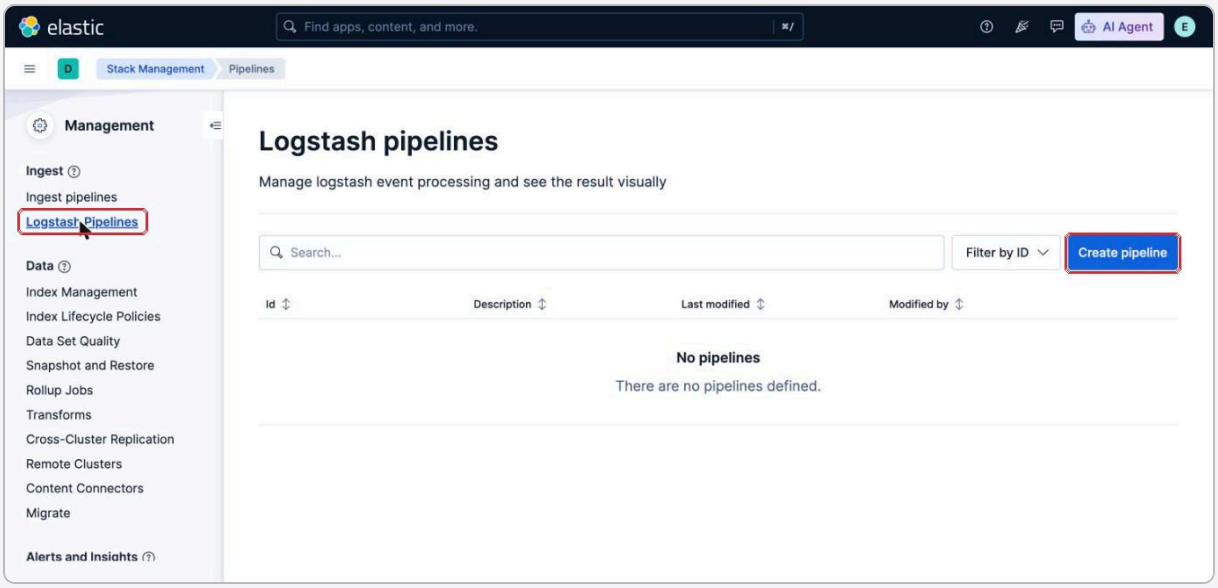
```
input {
  beats {
    port => 5044
  }
}

filter {
  grok {
    pattern_definitions => {
      "WHO"  => "%{USERNAME:user} %{IP:src_ip}"
      "WHAT" => "%{WORD:action} %{WORD:outcome} %{NUMBER:duration_ms:int}ms"
    }
    match => {
      "message" => "%{TIMESTAMP_ISO8601:ts} %{LOGLEVEL:level} %{WHO} %{WHAT}"
    }
  }
  date {
    match => ["ts", "ISO8601"]
    remove_field => ["ts"]
  }
  mutate {
    rename => {
      "level"   => "[log][level]"
      "user"    => "[user][name]"
      "src_ip"  => "[source][ip]"
      "action"  => "[event][action]"
      "outcome" => "[event][outcome]"
    }
  }
}

output {
  elasticsearch {
    hosts    => "elasticsearch:9200"
    user     => "logstash_internal"
    password => "${LOGSTASH_INTERNAL_PASSWORD}"
    index    => "logstash-applog"
  }
}
```

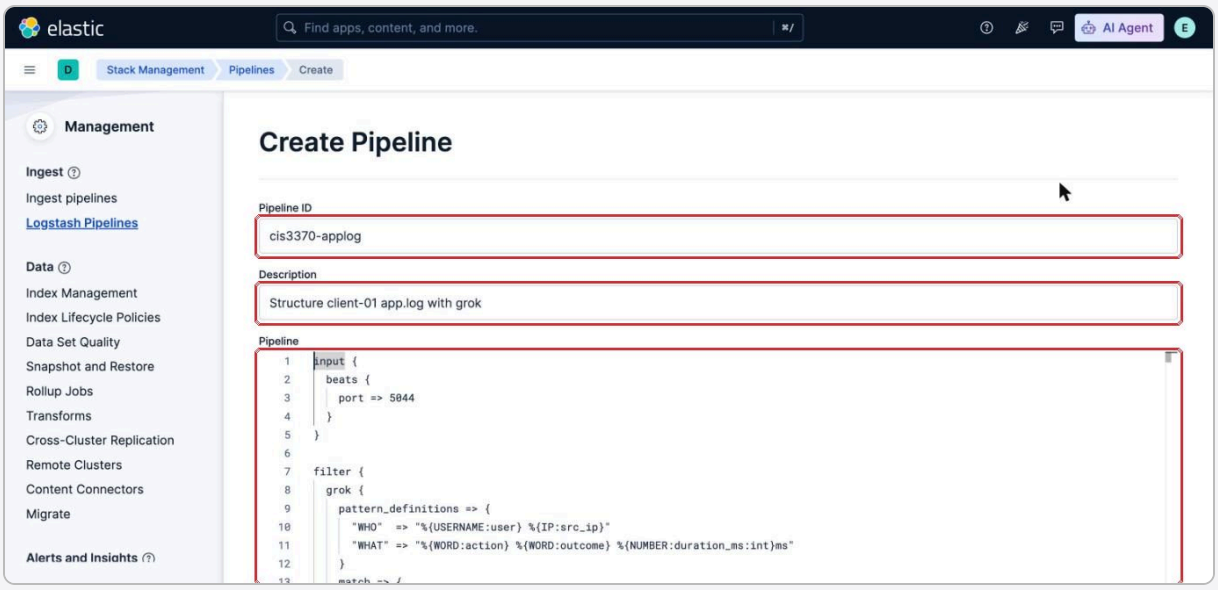
KIBANA UI

Stack Management → Ingest → Logstash Pipelines → Create pipeline.



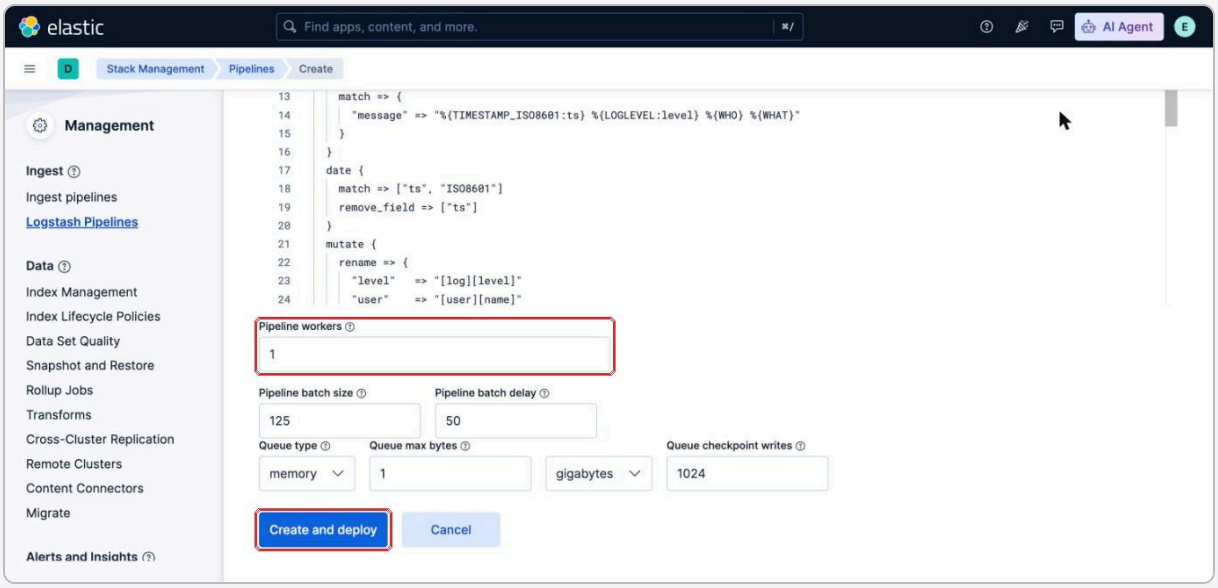
Logstash Pipelines, under Ingest. Empty until you create one.

Pipeline ID `cis3370-applog`, exactly the ID in `logstash.yml`. Select the template text in the Pipeline box and paste the pipeline over it.

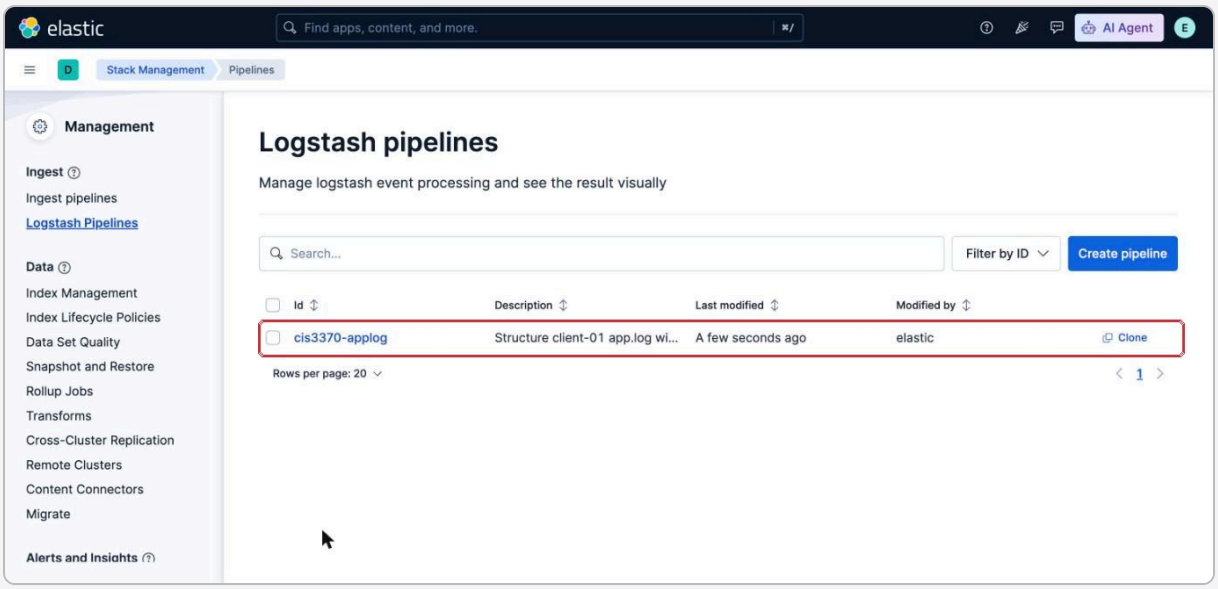


Pipeline ID, description, and the pipeline text.

Set **Pipeline workers** to `1`; a small log needs no more. Then **Create and deploy**.



Pipeline workers 1, then Create and deploy.



The pipeline is stored. Logstash picks it up at its next poll.

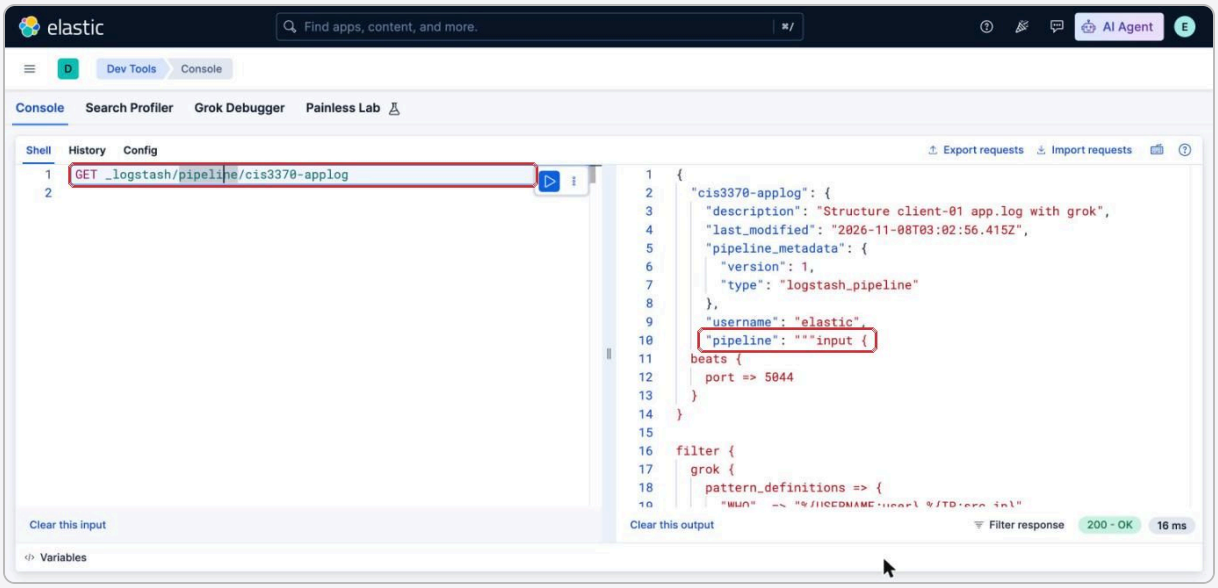
CONSOLE

The Console accepts triple quotes, `"""`, around text that runs over several lines, so the pipeline can be pasted as it is. A stored pipeline also needs its settings, a little metadata, a time and a user name:

```
PUT _logstash/pipeline/cis3370-applog
{
  "description": "Structure client-01 app.log with grok",
  "pipeline": """input {
  beats {
    port => 5044
  }
}
...the rest of the pipeline...""",
  "pipeline_settings": { "pipeline.workers": 1 },
  "pipeline_metadata": { "type": "logstash_pipeline", "version": 1 },
  "last_modified": "2026-01-01T00:00:00.000Z",
  "username": "elastic"
}
```

The answer is `201 - Created` with an empty body. `GET` shows what is stored:

```
GET _logstash/pipeline/cis3370-applog
```



The stored pipeline, with the user who saved it.

CURL

`HOST` Save the pipeline text as `applog.conf`. JSON cannot hold the line breaks as they are, so let Python, already on Ubuntu and macOS, build the body:

```
python3 - applog.conf > /tmp/applog.json <<'EOF'
import json, sys
print(json.dumps({
    "description": "Structure client-01 app.log with grok",
    "pipeline": open(sys.argv[1]).read(),
    "pipeline_settings": {"pipeline.workers": 1},
    "pipeline_metadata": {"type": "logstash_pipeline", "version": 1},
    "last_modified": "2026-01-01T00:00:00.000Z",
    "username": "curl"}))
EOF
curl -s -w "%{http_code}\n" -H "$KEYFILE" \
  -H 'Content-Type: application/json' -X PUT \
  "$ES/_logstash/pipeline/cis3370-applog" --data-binary @/tmp/applog.json
```

```
201
```

HOST Logstash starts it within five seconds:

```
docker compose logs --since 1m logstash | grep cis3370-applog
```

```
... Starting input listener {address: "0.0.0.0:5044"}
... Pipeline started {"pipeline.id" => "cis3370-applog"}
```

5 Ship app.log through Logstash

Filebeat may have only one output, and the Filebeat from Lab A already sends to Elasticsearch. So start a second Filebeat with its own small configuration and its own registry folder. Lab A's Filebeat keeps running untouched.

CLIENT-01

```
cat > /etc/filebeat/to-logstash.yml <<'EOF'
filebeat.inputs:
  - type: filestream
    id: applog
    paths:
      - /home/ubuntu/data/lab/app.log

output.logstash:
  hosts: ["logstash:5044"]
EOF
C="-c /etc/filebeat/to-logstash.yml --path.data /var/lib/filebeat-ls"
filebeat test config $C
filebeat test output $C
nohup filebeat -e $C > /var/log/filebeat-ls.log 2>&1 &
```

```
Config OK
logstash: logstash:5044...
  ...
  talk to server... OK
```

No password here: Logstash's beats input accepts any sender on the Docker network. That is fine inside this lab and is the next thing to fix anywhere real.

Count, then look closer

CONSOLE

```
GET logstash-applog/_count

GET logstash-applog/_mapping/field/source.ip,user.name
```

CURL

```
curl -s -H "$KEYFILE" "$ES/logstash-applog/_count"
curl -s -H "$KEYFILE" \
  "$ES/logstash-applog/_mapping/field/source.ip,user.name?pretty"
```

30 documents. Each one has `user.name`, `source.ip`, `event.action`, `event.outcome`, `log.level` and `duration_ms`, and its `@timestamp` is the time written in the line. But the mapping is wrong:

```
"source.ip" : { "mapping" : { "ip" : { "type" : "text", "fields" : {
"keyword" : ...
```

`source.ip` was stored as `text`, not `ip`. Ask how many logins failed per user, and Elasticsearch refuses:

```
Fielddata is disabled on [event.outcome] in [logstash-applog]. Text fields
are not optimised for
operations that require per-document field data like aggregations and sorting
...
```

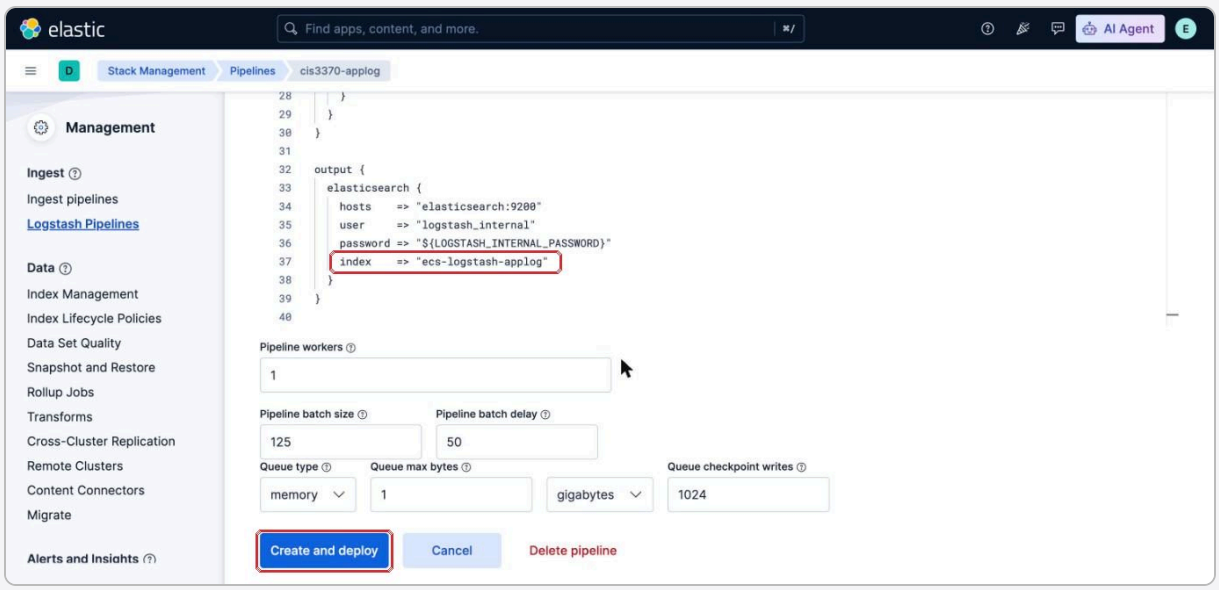
Nobody told Elasticsearch what these fields are, so it guessed from the first document: every string became `text`. Grok gave the pieces names; only an index template gives them types.

6 Fix the mapping

Logstash installs an index template called `ecs-logstash` for every index whose name starts with `ecs-logstash-`. It knows the ECS fields: `source.ip` is an address, `user.name` and `event.outcome` are keywords. So the fix is one word in the output: the index name.

KIBANA UI

Logstash Pipelines, click `cis3370-applog`, change the last line of the output to `index => "ecs-logstash-applog"`, and deploy again.



One line changes: the index name.

CONSOLE

Run the same `PUT _logstash/pipeline/cis3370-applog` with the new index line. A `PUT` replaces what was stored.

CURL

Change the line in `applog.conf`, rebuild `/tmp/applog.json` with the same Python command, and send the same `PUT`.

Logstash reloads it by itself (`Reloading pipeline`, then `Pipeline started`). Now send the file again. The second Filebeat remembers it already sent all 30 lines, so stop it, make it forget, and start it again.

CLIENT-01

```
pkill -f to-logstash.yml
rm -rf /var/lib/filebeat-ls/registry
C="-c /etc/filebeat/to-logstash.yml --path.data /var/lib/filebeat-ls"
nohup filebeat -e $C > /var/log/filebeat-ls.log 2>&1 &
```

Validate three ways

CONSOLE

```
GET ecs-logstash-applog/_search?filter_path=hits.total,aggregations
{
  "size": 0,
  "aggs": {
    "outcome": { "terms": { "field": "event.outcome" } },
    "failed_by": {
      "filter": { "term": { "event.outcome": "failure" } },
      "aggs": { "user": { "terms": { "field": "user.name" } } }
    },
    "avg_ms": { "avg": { "field": "duration_ms" } },
    "lab_net": { "filter": { "term": { "source.ip": "172.21.0.0/16" } } }
  }
}
```

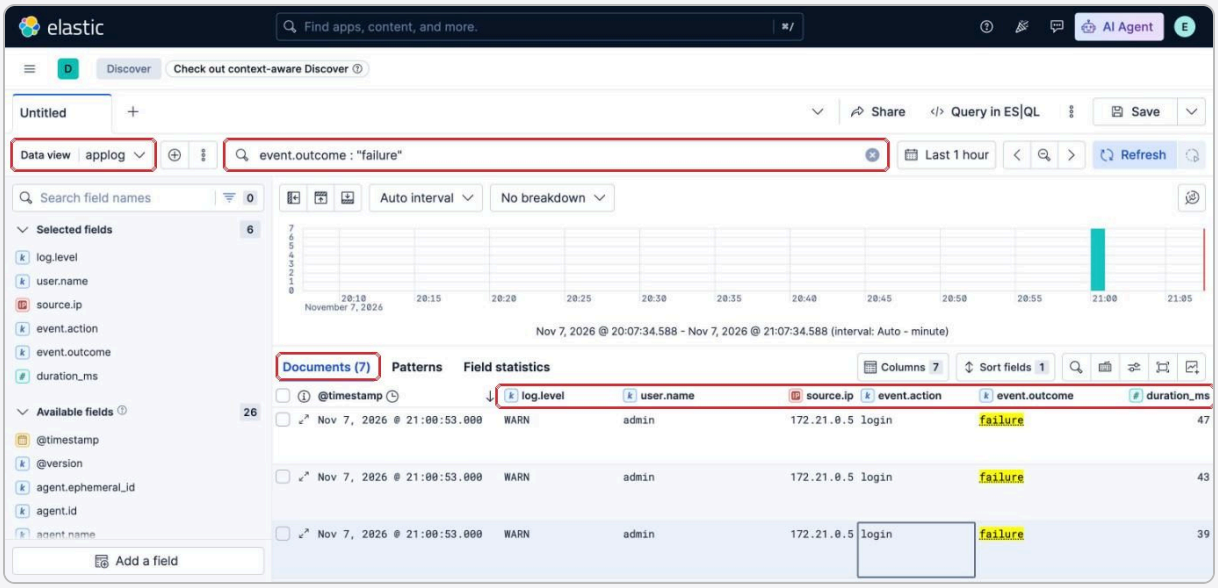
CURL

```
curl -s -H "$KEYFILE" "$ES/ecs-logstash-applog/_count"
curl -s -H "$KEYFILE" \
  "$ES/ecs-logstash-applog/_mapping/field/source.ip?pretty"
```

CHECK	RESULT ON THE CLASS STACK
Documents	30
`source.ip` type	`ip`
`user.name`, `event.outcome`, `log.level`	`keyword`
`duration_ms`	`long`, average 35.5
Outcomes	23 success, 7 failure
Failures by user	`admin` 7
In `172.21.0.0/16`	30, a search that only works on an `ip` field

DISCOVER

Create a data view named `applog` for `ecs-logstash-applog`, as in Lab A, then search `event.outcome : "failure"` and add the six fields as columns.



Seven failures, all admin, every piece in its own typed field.

7 Put Logstash back

Restore docker-elk's own pipeline so the rest of the course finds Logstash as it was.

CLIENT-01

```
pkill -f to-logstash.yml
```

HOST

```
cp logstash/config/logstash.yml.original logstash/config/logstash.yml
docker compose restart logstash
curl -s -H "$KEYFILE" -X DELETE "$ES/_logstash/pipeline/cis3370-applog"
curl -s -H "$KEYFILE" -X DELETE "$ES/logstash-applog"
curl -s -H "$KEYFILE" -X DELETE "$ES/ecs-logstash-applog"
curl -s -H "$KEYFILE" -X DELETE "$ES/_security/user/logstash_cpm"
curl -s -H "$KEYFILE" -X DELETE "$ES/_security/role/logstash_cpm"
```

Delete the `applog` data view in **Stack Management → Data Views**.

8 Questions

QUESTION 1

Grok gave every piece a name, yet the first index could not count failures per user. Explain the difference between naming a field and typing it, using the error you saw.

QUESTION 2

Find one search that works on `ecs-logstash-applog` and fails or gives a wrong answer on `logstash-applog`, other than the one in this lab. Show both results.

QUESTION 3

Logstash refused to send an API key over HTTP but happily sends a password over HTTP. Which is the bigger risk on this network, and what is the real fix?

QUESTION 4

Why is the `logstash_cpm` account limited to `manage_logstash_pipelines`? What could an attacker who stole it change, and what could they not?

QUESTION 5

Write a line `app.log` might contain that your pattern does not match. What does Logstash do with it? Find out with the debugger, then by appending it.

9 What to hand in

1. Your Grok Debugger screenshot with the structured result.
2. The Logstash log lines showing `No configuration found`, then `Pipeline started`.
3. The mapping of `source.ip` in both indices, and the error from the first one.
4. The table from section 6 with your numbers, and your Discover screenshot.
5. Your answers to the five questions.

Every number and screenshot on this page comes from running these steps on the class stack: Logstash, Elasticsearch, Kibana and Filebeat 9.5.3. Times and identifiers differ on your machine; the counts should not.